Universität Konstanz

Applied Natural Language Processing (NLP) for Online Harm Detection

Jun.-Prof. Dr. Miriam Schirmer

Konstanz, 12.11.2026

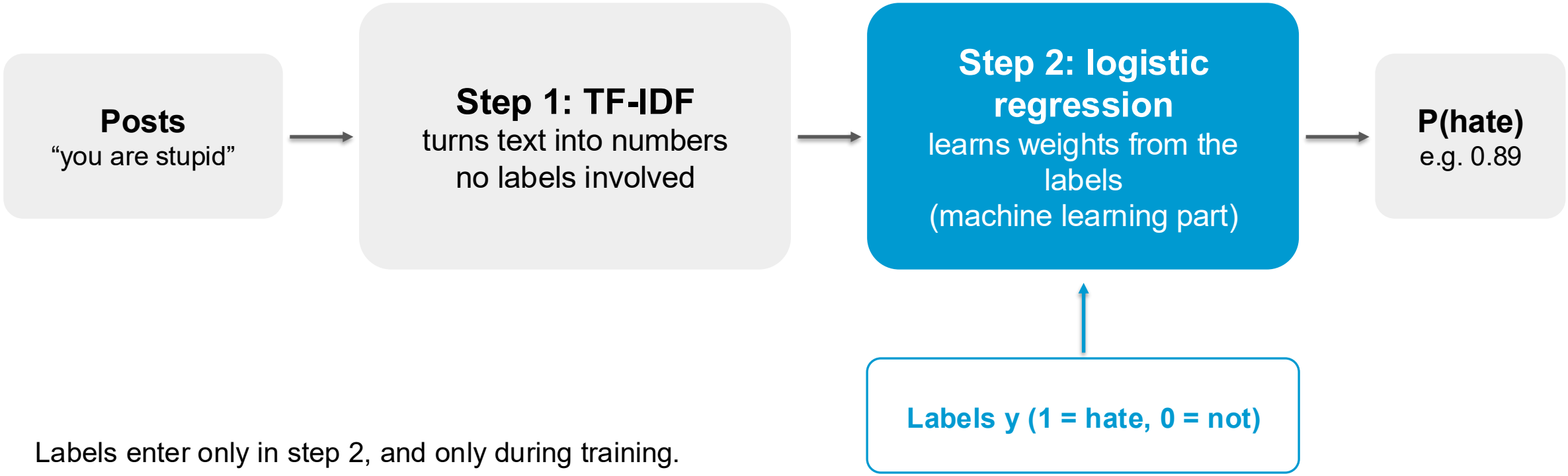
Outline for Today

Session 4: Embeddings, Transformers and LLMs

- Recap: TF-IDF and logistic regression
- Readings: J&M ch. 7 and Plaza-del-Arco et al. (2023)
- Embeddings and neural networks
- Transformers and attention
- Encoders, decoders and LLMs
- Mini-notebook: an open-source LLM as annotator

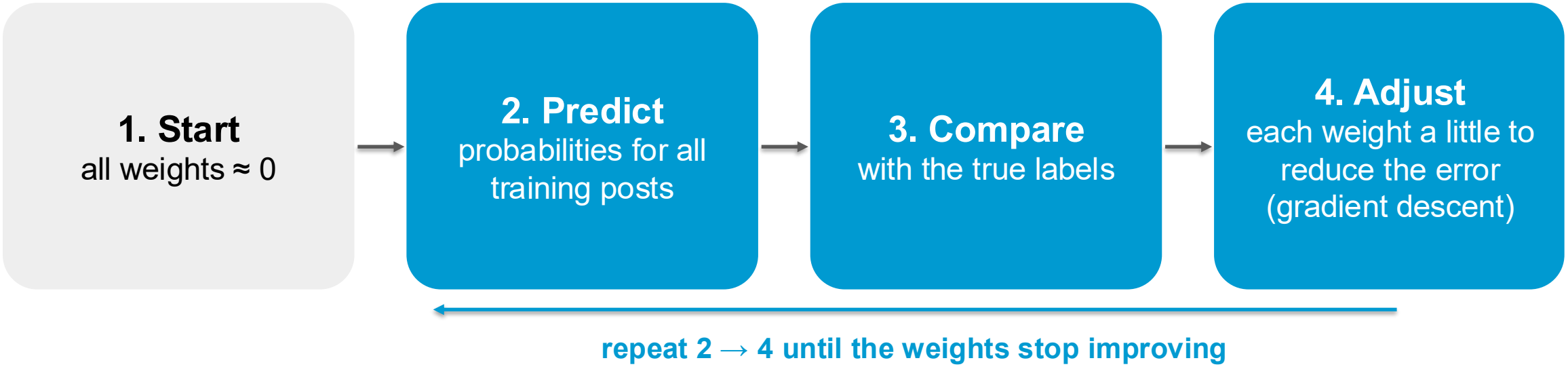
Recap: Text as Features for Classification

Two Step Pipeline



Labels enter only in step 2, and only during training.

Training

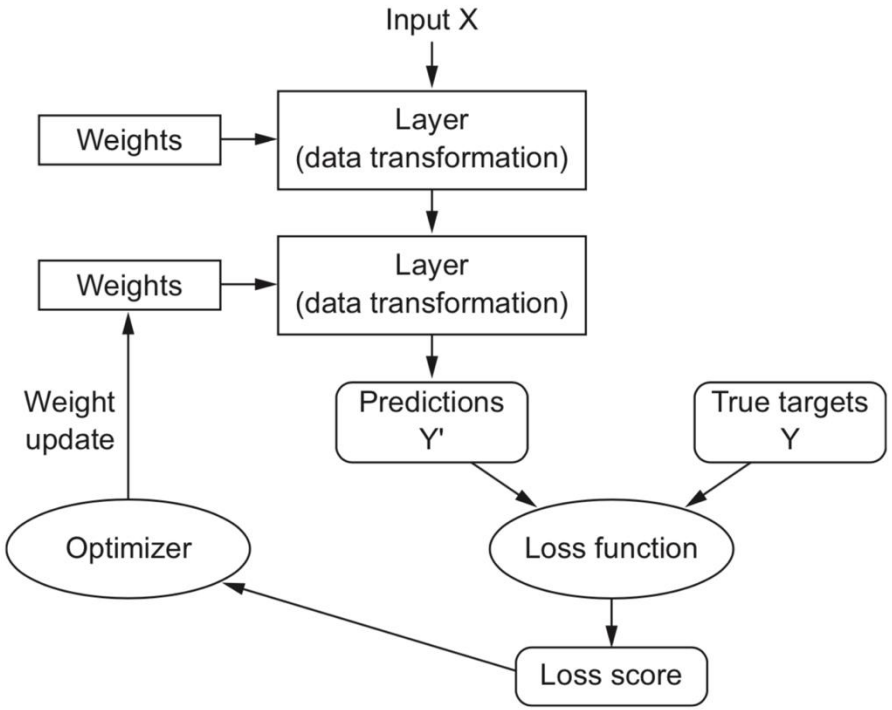


- Once training ends, the weights stay fixed.
- TF-IDF values never change during training.

Training Loop

In our training data, we know the correct label *y* (0 or 1) for each *x*.

1. Start
2. The **weights** produce predictions
3. We want to set *w* and *b* to **minimize the distance** between our estimate $\hat{y}$ and the true *y*
   - We estimate this distance with a **loss function** (a.k.a. cost function); it compares predictions and true labels

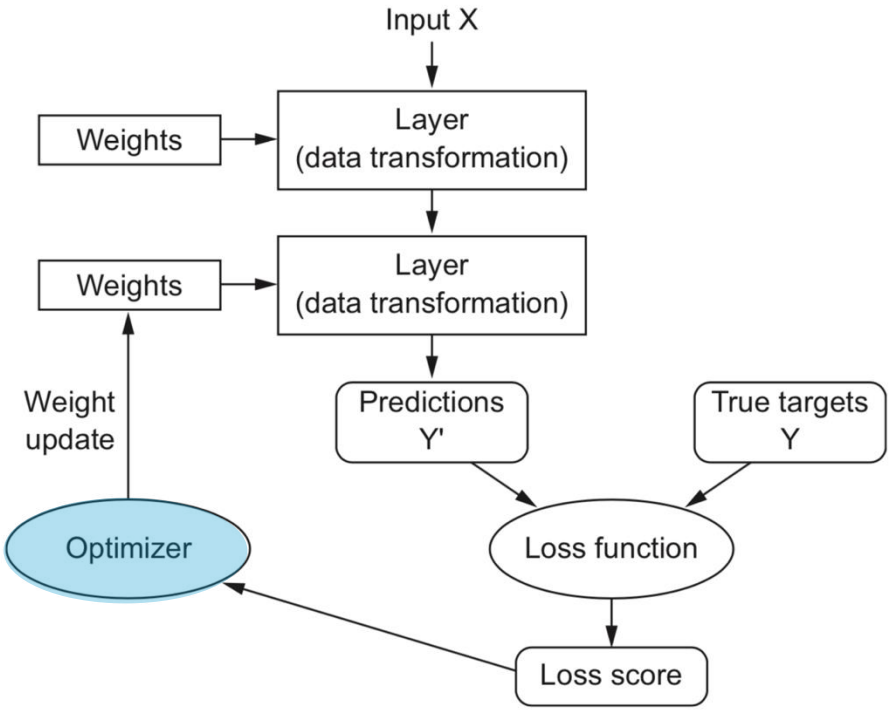


Deep Learning with Python: https://deeplearningwithpython.io

Training Loop

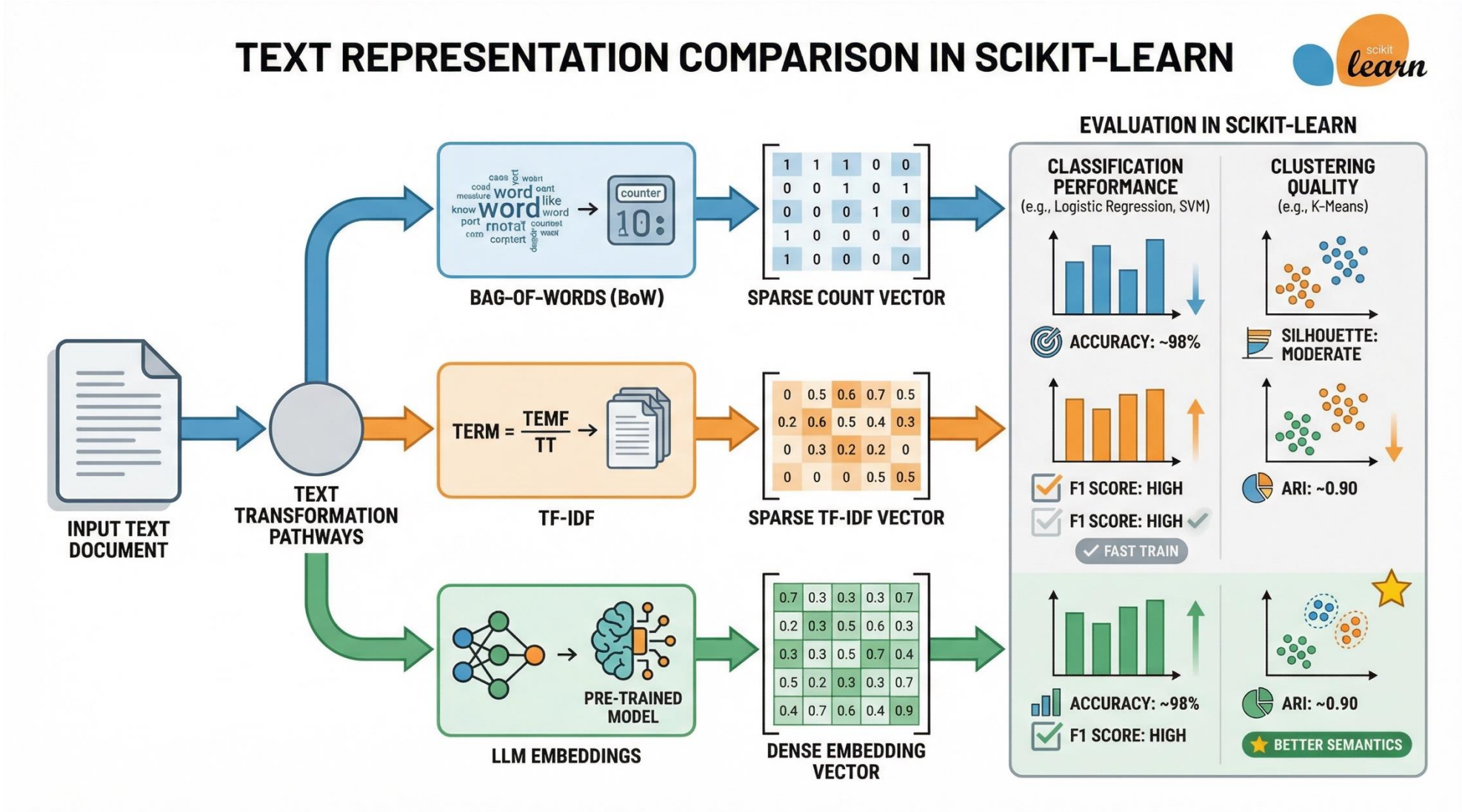
We need an **optimization algorithm** to update w and b to minimize the loss.

It updates the weights to lower the loss and repeats until the loss stops improving (stochastic gradient descent)



Deep Learning with Python: https://deeplearningwithpython.io

Note: we're focusing on binary classification for simplicity. Multi-class (multinomial) logistic regression has the same logic but slightly different math.



https://machinelearningmastery.com/wp-content/uploads/2026/02/mlm-chugani-llm-embeddings-vs-tf-idf-vs-bag-of-words-works-better-scikit-learn-feature.jpg

To Explain or to Predict?

In a TF-IDF logistic regression model, every word in the vocabulary (or bigram, if we include them) is a predictor

- More predictors than posts → many different weight combinations fit the training posts perfectly

This can be a problem for two reasons:

- Statistics: **no unique solution**, so the model cannot be estimated
- Machine learning: the model uses rare words to get each training post right → it memorizes the posts instead of learning patterns (**overfitting**)

Shmueli, G. (2010). To explain or to predict? *Statistical Science, 25*(3), 289–310, https://doi.org/10.1214/10-STS330.

To Explain or to Predict?

Solutions

- Use fewer words (e.g., only the most frequent ones)
- **Regularization!** → keep all words, but penalize big weights (a word needs evidence from many posts to matter)
  - This prevents that correlated words can get large weights but point in different directions

Inference statistics vs. machine learning:
In ML, we care more about predictions and evaluate on a test set (F1), vs. inference statistics relies on interpretable coefficients and, for example, p-values for evaluation.

Shmueli, G. (2010). To explain or to predict? *Statistical Science, 25*(3), 289–310, https://doi.org/10.1214/10-STS330.

Let’s Talk About the Readings!

Let's Talk About the Readings

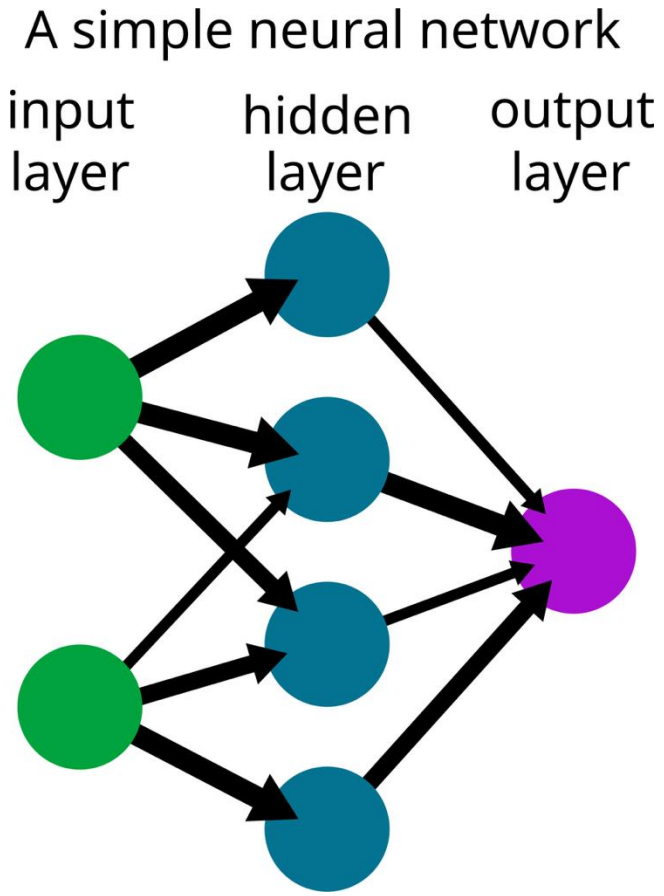
- **Jurafsky & Martin, ch. 7:** what can a transformer do with the word "it" that word2vec cannot?
- **Plaza-del-Arco, Nozza & Hovy (2023):** how can a language model detect hate speech without labeled training data, and what makes the biggest difference?

Neural Networks in Ten Minutes

What Is a Neural Network?

- Turns numbers in (a post) into numbers out (P(hate)) (in our application case)
- \
- Made of simple **units** in **layers**: input → hidden → output
- Each unit: weighted sum, then a non-linear function, like logistic regression
- Each arrow is a **weight**, learned with loss + gradient descent

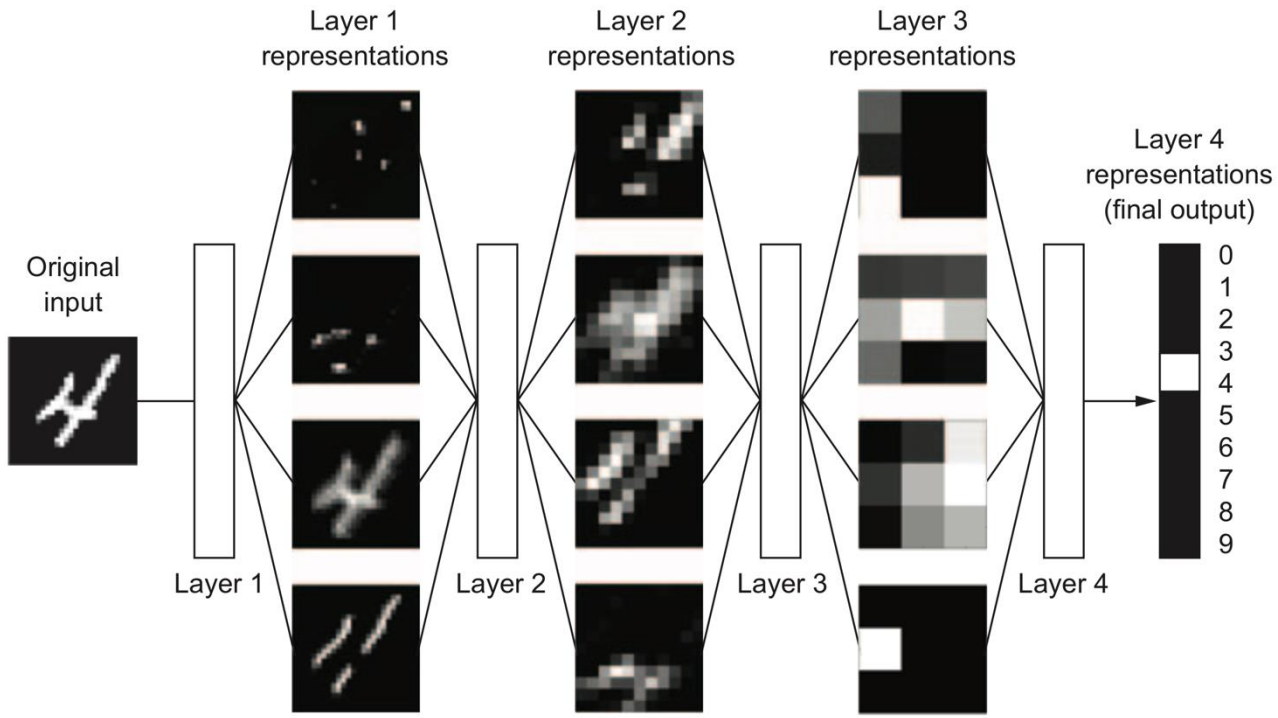
- Logistic regression = the smallest neural network: one unit, no hidden layer



Wiso (2008). Neural network example. Wikimedia Commons (public domain)

What Is a Hidden Layer?

- Any layer between input and output: we never see or define what it computes
- It turns its input into **new features** that help the next layer
- Images: early layers find strokes, later layers whole shapes



https://deeplearningwithpython.io

Why Neural Networks for Text?

- TF-IDF + logistic regression: **we** choose the features (one per word)
- Meaning depends on order and combinations (“not bad”, “go back to …”): hard to hand-code
- Neural networks **learn** these features from data, even from text without labels

- word2vec, BERT, GPT and Qwen are all neural networks
- Qwen2.5-1.5B (we will work with this later): 1.5 billion weights

Different Kinds of Neural Networks

Feedforward (MLP)
How: layer after layer
Used for: a classifier on top of embeddings

Convolutional (CNN)
How: small filters slide over the input
Used for: images

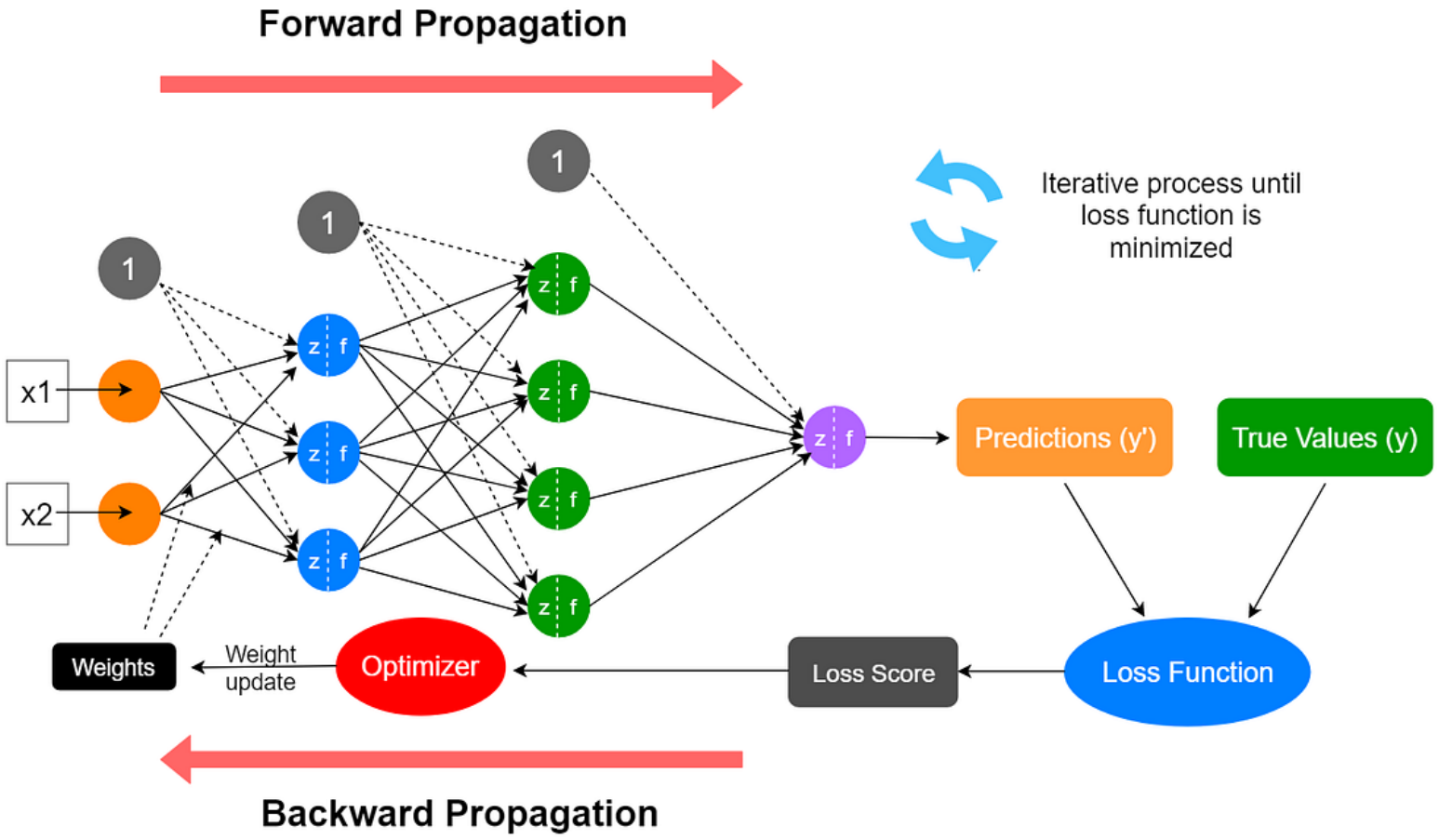
Recurrent (RNN, LSTM)
How: reads word by word, with a memory
Used for: text before 2017

Transformer
How: every word looks at every other word (attention)
Used for: BERT, GPT, Qwen

Training a Network: Forward, Loss, Backward

The entire learning process can be divided into three main parts:

1. **Forward propagation (Forward pass)**
2. **Calculation of the loss function**
3. **Backward propagation (Backward pass/Backpropagation)**



https://medium.com/data-science-365/overview-of-a-neural-networks-learning-process-61690a502fa

Embeddings

What Does *ongchoi* Mean?

Suppose you see these sentences

- Ongchoi is delicious sautéed with garlic
- Ongchoi is superb over rice
- Ongchoi leaves with salty sauces

And you have also seen

- … spinach sautéed with garlic over rice
- Chard stems and leaves are delicious

Conclusion

- Ongchoi is a leafy green like spinach or chard



https://de.wikipedia.org/wiki/Wasserspinat#/media/Datei:Kangkungblacan.jpg

Adapted from Jurafsky, D., & Martin, J. H. (2026). Speech and Language Processing (3rd ed. draft).

You Shall Know a Word by the Company It Keeps

Distributional hypothesis

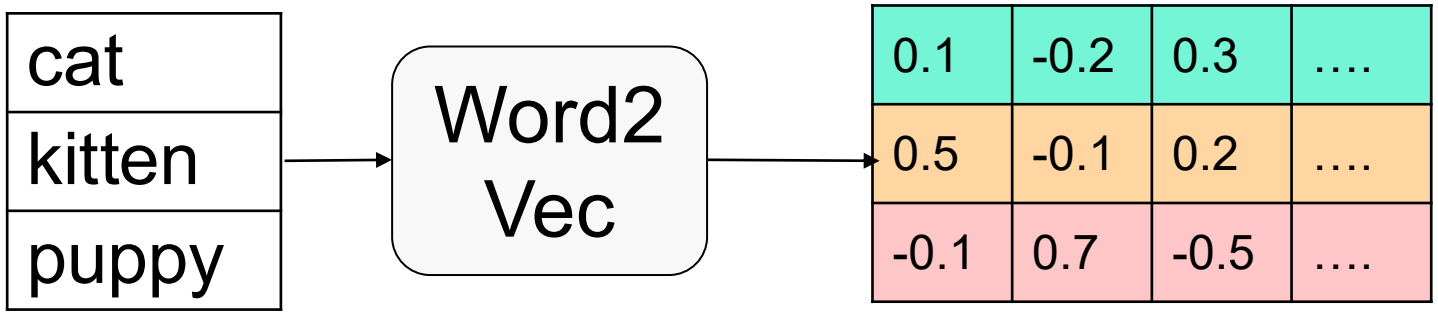
- Words with similar meanings are used in similar contexts
- “You shall know a word by the company it keeps” (Firth 1957)

What counts as context?

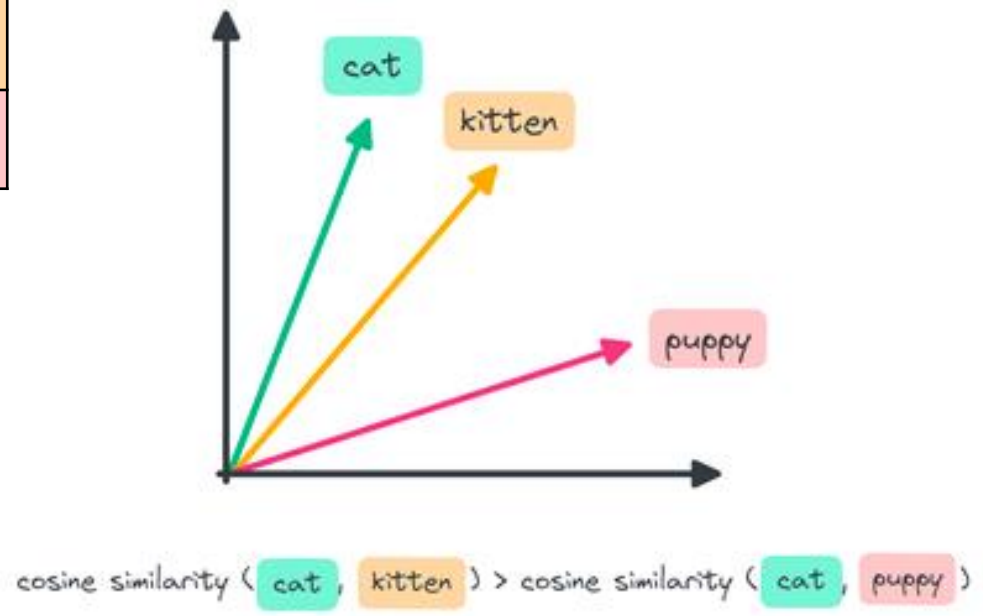
- **TF-IDF:** the documents a word appears in
- **word2vec:** the words nearby (a window of ±2 to ±5 words)

- From many contexts of a word, word2vec learns one dense vector: its **embedding**

word2vec: Similar Contexts, Similar Vectors



$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|}$$



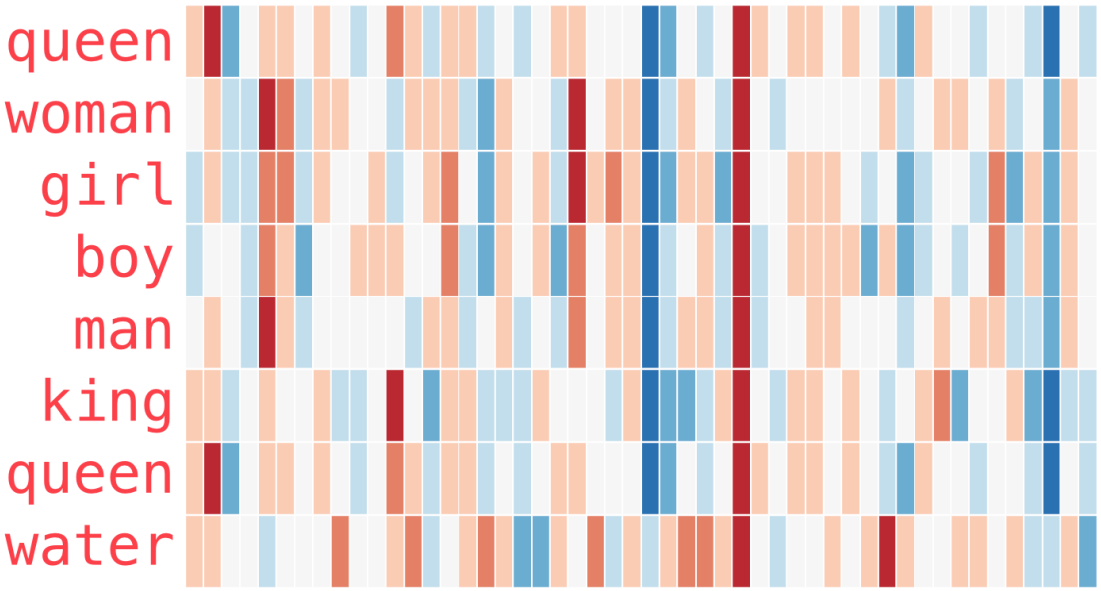
High **cosine similarity** → words occur in similar contexts → share some similar meanings

https://techcommunity.microsoft.com/blog/educatordeveloperblog/building-a-vector-similarity-search-app-with-azure-ai-vision-and-postgresql/3954936

How Many Dimensions Does an Embedding Have?

The word “king” as 50 numbers (GloVe, "Global Vectors", Stanford 2014)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49
0	0.5	0.69	-0.6	-0.023	0.6	-0.13	-0.088	0.47	-0.62	-0.31	-0.077	1.5	-0.034	-0.98	0.68	0.82	-0.52	-0.32	-0.56	0.66	0.2	-0.13	-0.11	-0.3	0.41	-2.2	-1.1	-1.1	-0.34	0.34	2	-0.042	-0.64	0.71	0.49	0.17	0.34	-0.26	-0.85	0.17	0.4	1.2	-1	-0.22	-0.15	0.78	-0.91	-1.6	-0.64	-0.51

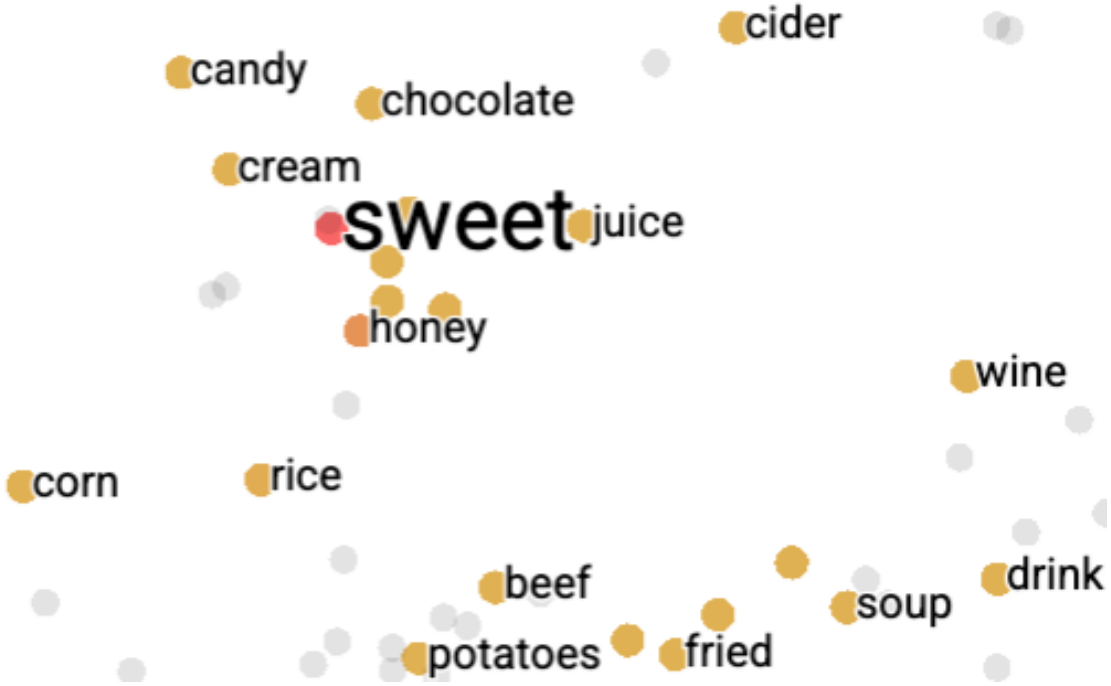


Each number is one **dimension**
Typical sizes:
word2vec, GloVe: 50 to 300
BERT: 768; sentence-transformers (MiniLM): 384
Qwen2.5-1.5B (today): 1,536; large LLMs: several thousand
Single dimensions mean nothing on their own; similar words show **similar patterns**

Alammar, J. (2019). The illustrated word2vec. jalammar.github.io/illustrated-word2vec (CC BY-NC-SA 4.0)

Meaning as a Point in Space

- Each word is a vector, not just a symbol
- Similar words are nearby in this space
- The space is built automatically from which words occur near each other in text



Adapted from Jurafsky, D., & Martin, J. H. (2026). Speech and Language Processing (3rd ed. draft).

Seeing Embeddings in Two Dimensions

- We cannot look at 300 dimensions, so we squeeze them into 2 for a plot
- **PCA:** keeps the 2 directions in which the words differ most
- **t-SNE / UMAP:** keep neighbors close together; good for seeing clusters
- Example: words from 19th-century novels; similar words form clusters

- Only for looking, not for measuring: distances in the 2D picture can mislead, so compute similarities in the full space

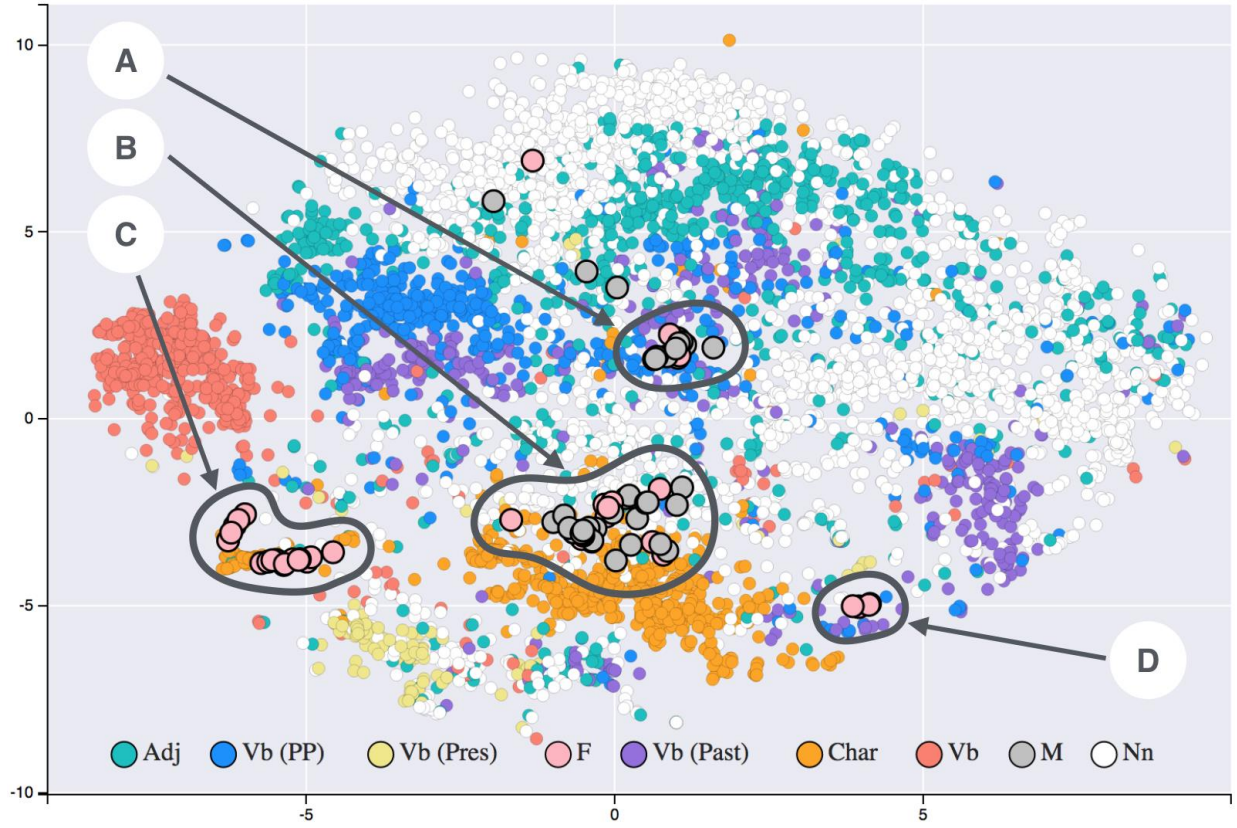


Figure: Grayson, S. (2017). t-SNE visualisation of word embeddings generated using 19th century literature. Wikimedia Commons (CC BY-SA 4.0)

Static: One Point per Word

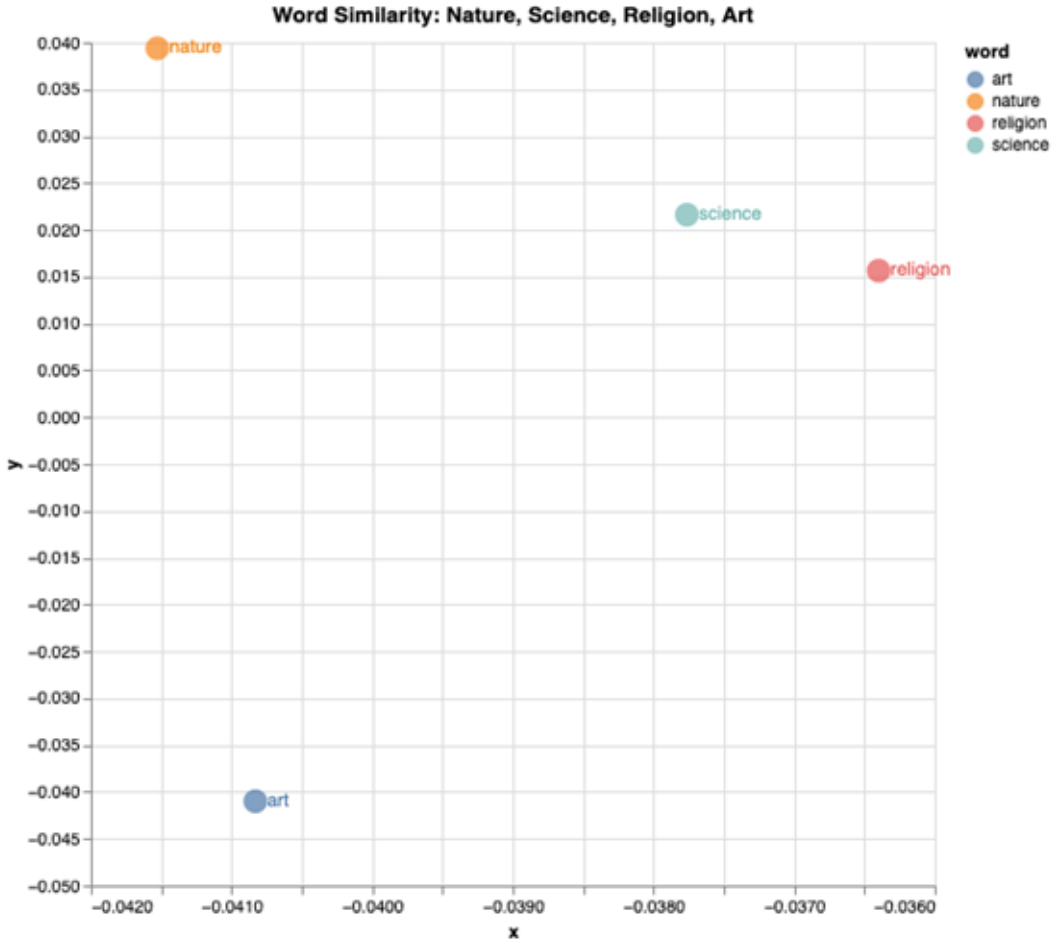


Figure: Mendelsohn, J. (2026). INST664, Week 3: Embeddings. University of Maryland (course slides).

Contextual (BERT): One Point per Word in Context

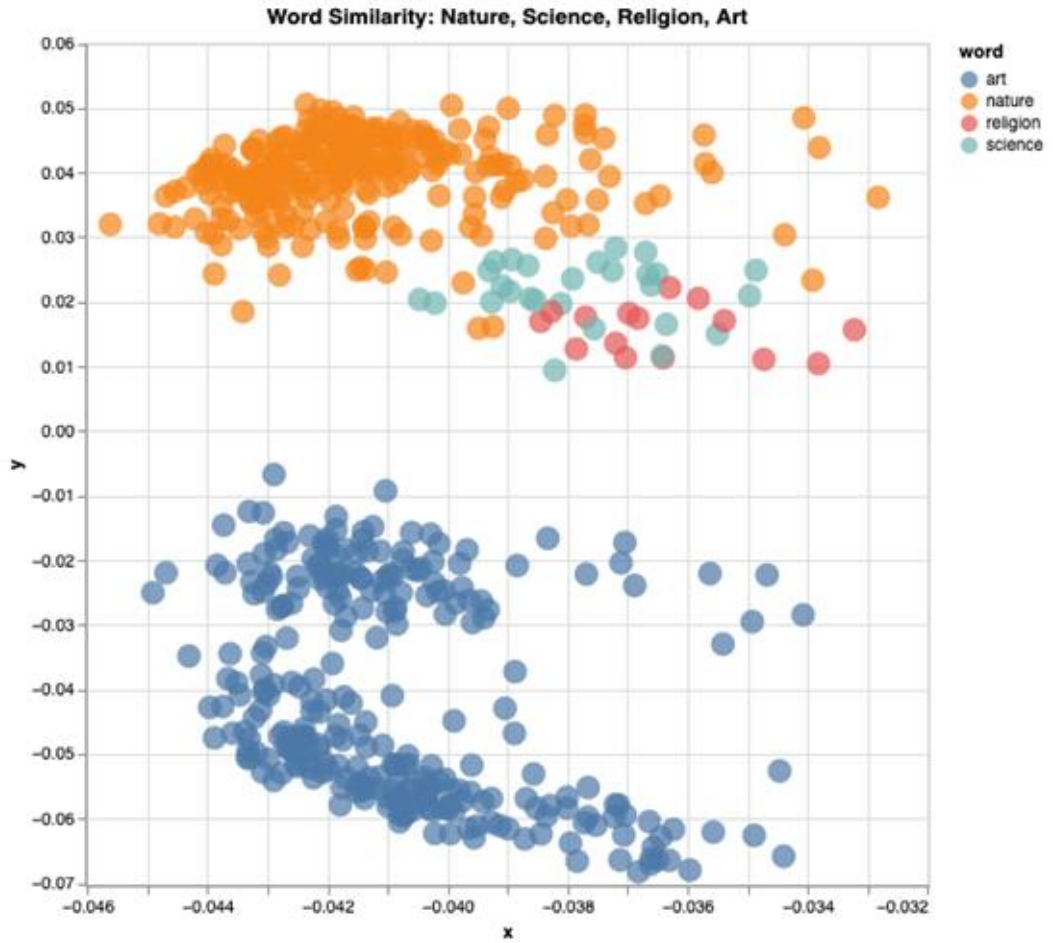


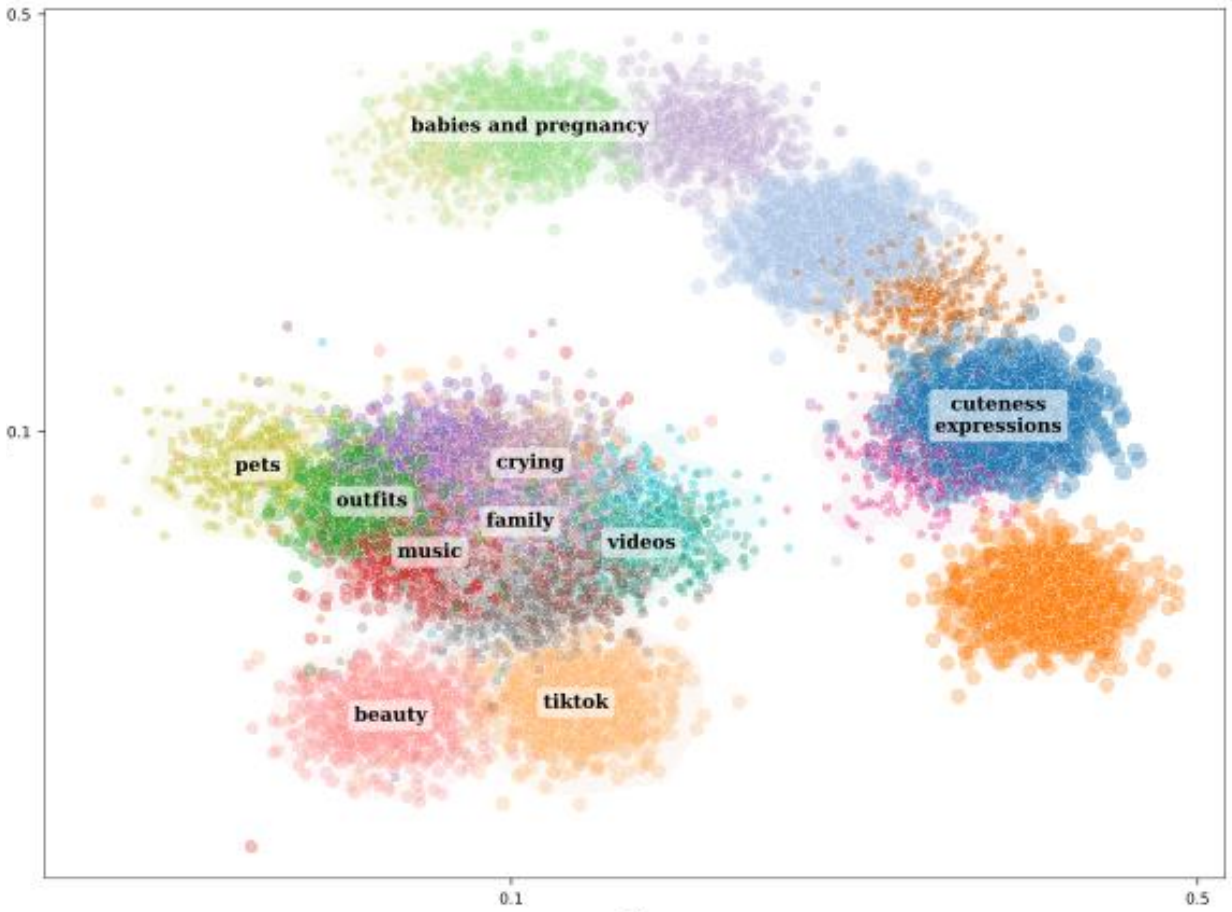
Figure: Mendelsohn, J. (2026). INST664, Week 3: Embeddings. University of Maryland (course slides).

word2vec vs. BERT

	word2vec (2013)	BERT (2018)
What it is	a small neural network (one hidden layer)	a large neural network (transformer, 12 layers)
Learns from	which words appear near each other	filling in masked words, using the whole sentence
Output	one fixed vector per word (static)	a new vector for each word in each sentence (contextual)
“bank”	same vector in “river bank” and “bank account”	different vectors
Typical use	word associations (Garg et al.), expanding dictionaries	classifying texts, fine-tuning, BERTopic

Both are neural networks that learn from raw text without labels, and both give you embeddings.

Example



Language Context:
What do people talk about in child-related comments on TikTok?
(Schirmer et al., 2025)

BERT (Devlin et al., 2018)



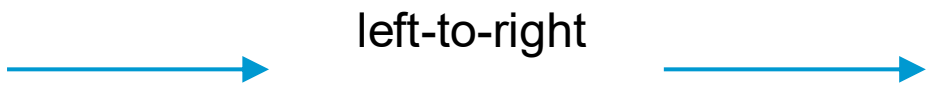
Bidirectional

Encoder

Representations from

Transformers

Bidirectional



“NLP's ImageNet moment has [MASK]”



“[MASK] ImageNet moment has arrived ”



“NLP's ImageNet [MASK] has [MASK]”

What Does Fine-Tuning Mean?

- Start from a model that is **already trained** on huge amounts of text (pre-trained)
- Add a small classification layer on top
- **Continue training** on your own, much smaller labeled data (e.g. 2,000 posts): same loss and gradient descent as before
- All weights are nudged only **a little**: the model keeps its knowledge of language and adapts to your task



Language-Specific BERT Models









Domain-Specific BERT Models

FinBERT - Finance

Training data: Financial news

ClinicalBERT - Clinical/medical notes

Training data: Electronic health records

SciBERT - Scientific publications

Training data: 1.14M Semantic Scholar papers

LegalBERT -Legal documents

Training data: Harvard Law case corpus from 1965

BERTweet -Social media (Twitter)

Training data: Large-scale English tweets

How Can We Use Embeddings in Research?

Embeddings Reflect Cultural Bias

- Paris : France :: Tokyo : x → x = Japan
- father : doctor :: mother : x → x = nurse
- man : computer programmer :: woman : x → x = homemaker

- A problem for applications (e.g. hiring), but also a **measurement tool**

Bolukbasi, T., Chang, K.-W., Zou, J., Saligrama, V., & Kalai, A. (2016). Man is to computer programmer as woman is to homemaker? Debiasing word embeddings. NeurIPS.

Garg et al. (2018): 100 Years of Stereotypes

Can word embeddings measure how gender and ethnic stereotypes in the US changed since 1910?

1. Embeddings per decade
trained separately on text from each decade (books, newspapers)

→

2. Word lists
group words (she, woman, … vs. he, man, …; family names) and neutral words (occupations, adjectives)

→

3. Distance
how much closer is "nurse" to the women words than to the men words?

→

4. Validation
compare with the US Census: share of women in each occupation

Garg, N., Schiebinger, L., Jurafsky, D., & Zou, J. (2018). Word embeddings quantify 100 years of gender and ethnic stereotypes. PNAS, 115(16), E3635–E3644.

1910	1920	1930	1940	1950	1960	1970	1980	1990
honorable	regimental	honorable	honorable	knowledge	gallant	honorable	honorable	honorable
gallant	honorable	trusting	conservative	gallant	honorable	wise	loyal	regimental
regimental	stoic	courageous	ambitious	honorable	sage	knowledge	petty	unreliable
skillful	political	gallant	shrewd	directed	regimental	gallant	gallant	skillful
disobedient	sage	confident	regimental	regimental	knowledge	insulting	lyrical	gallant
faithful	ambitious	adventurous	knowledge	efficient	wise	trusting	honest	honest
wise	reserved	experimental	destructive	sage	conservative	honest	faithful	loyal
obedient	progressive	efficient	misguided	wise	honest	providential	obedient	wise
obnoxious	unprincipled	predatory	gallant	faithful	adventurous	modern	wise	directed
steadfast	shrewd	modern	petty	creative	efficient	regimental	hostile	courageous

Table B.29: Most Man adjectives in each decade in the SGNS embeddings.

1910	1920	1930	1940	1950	1960	1970	1980	1990
charming	charming	charming	delicate	delicate	sweet	attractive	maternal	maternal
placid	relaxed	delicate	placid	sweet	charming	maternal	attractive	morbid
delicate	delicate	soft	sweet	charming	soft	charming	masculine	artificial
passionate	amiable	hysterical	gentle	transparent	relaxed	sweet	impassive	physical
sweet	hysterical	transparent	soft	placid	attractive	caring	emotional	caring
dreamy	placid	sweet	warm	childish	placid	venomous	protective	emotional
indulgent	soft	relaxed	charming	soft	delicate	silly	relaxed	protective
playful	gentle	shy	childish	colorless	maternal	neat	charming	attractive
mellow	attractive	maternal	irritable	tasteless	indulgent	delicate	naive	soft
sentimental	sweet	smooth	maternal	agreeable	gentle	sensitive	responsive	tidy

Table B.30: Most Woman adjectives in each decade in the SGNS embeddings.

Garg et al. (2018), PNAS,
Supplementary Tables B.29–B.30

Transformers and Attention

One Vector per Word Is Not Enough

The same word can mean different things

- I went to the river **bank** yesterday
- I went to the **bank** to withdraw some money

What does “it” mean here?

- The chicken didn’t cross the road because **it** was too tired
- The chicken didn’t cross the road because **it** was too wide

- Static embeddings (word2vec) give “bank” and “it” the same vector every time. We need **contextual embeddings**

Adapted from Jurafsky, D., & Martin, J. H. (2026). Speech and Language Processing (3rd ed. draft).

LLMs Are Built Out of Transformers

- Transformer: a neural network architecture based on **attention** (Vaswani et al. 2017)

Attention Is All You Need

Ashish Vaswani[*]
Google Brain
avaswani@google.com

Noam Shazeer[*]
Google Brain
noam@google.com

Niki Parmar[*]
Google Research
nikip@google.com

Jakob Uszkoreit[*]
Google Research
usz@google.com

Llion Jones[*]
Google Research
llion@google.com

Aidan N. Gomez[*] [†]
University of Toronto
aidan@cs.toronto.edu

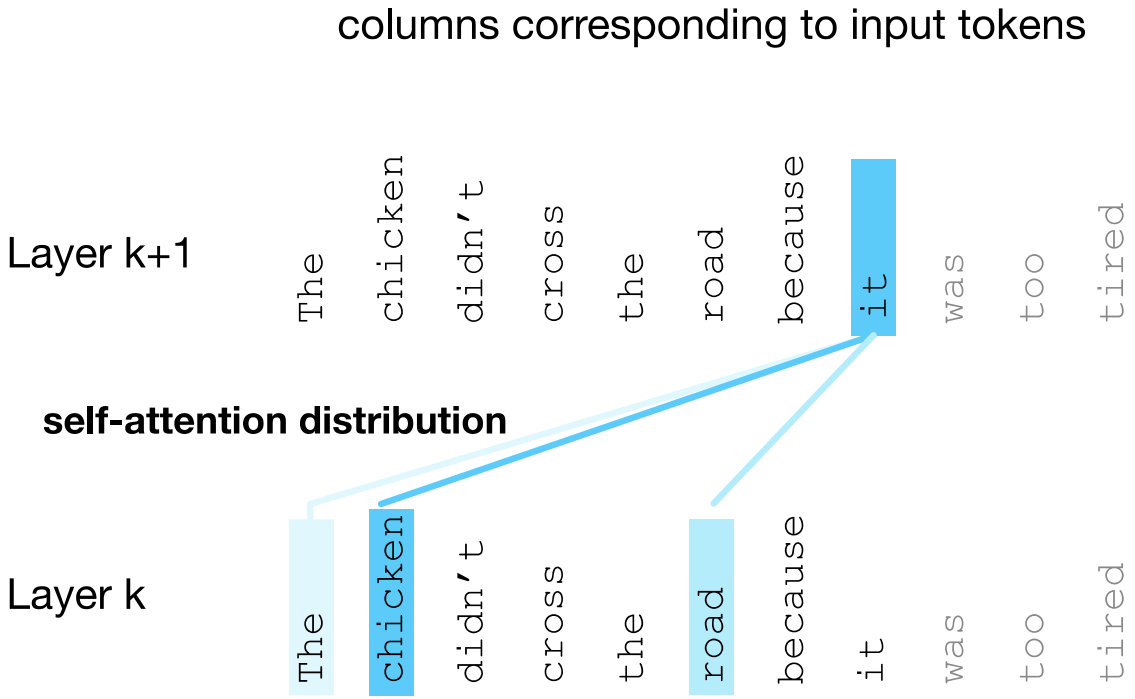
Łukasz Kaiser[*]
Google Brain
lukaszkaiser@google.com

Illia Polosukhin[*] [‡]
illia.polosukhin@gmail.com

Adapted from Jurafsky, D., & Martin, J. H. (2026). Speech and Language Processing (3rd ed. draft).

Intuition of Attention

- What does ”it” mean in this context?
- To build the vector for “it”, the model looks at all other words and mixes in information from the relevant ones



Adapted from Jurafsky, D., & Martin, J. H. (2026). Speech and Language Processing (3rd ed. draft).

Attention Without Formulas

- Each token starts with its static embedding (plus its position)
- It looks at all other tokens and scores **how relevant** each one is
- It takes a **weighted mix** of their information → a new, contextual vector
- Many attention “heads” do this in parallel, and many layers are stacked on top of each other
- Result: **one contextual vector per token**

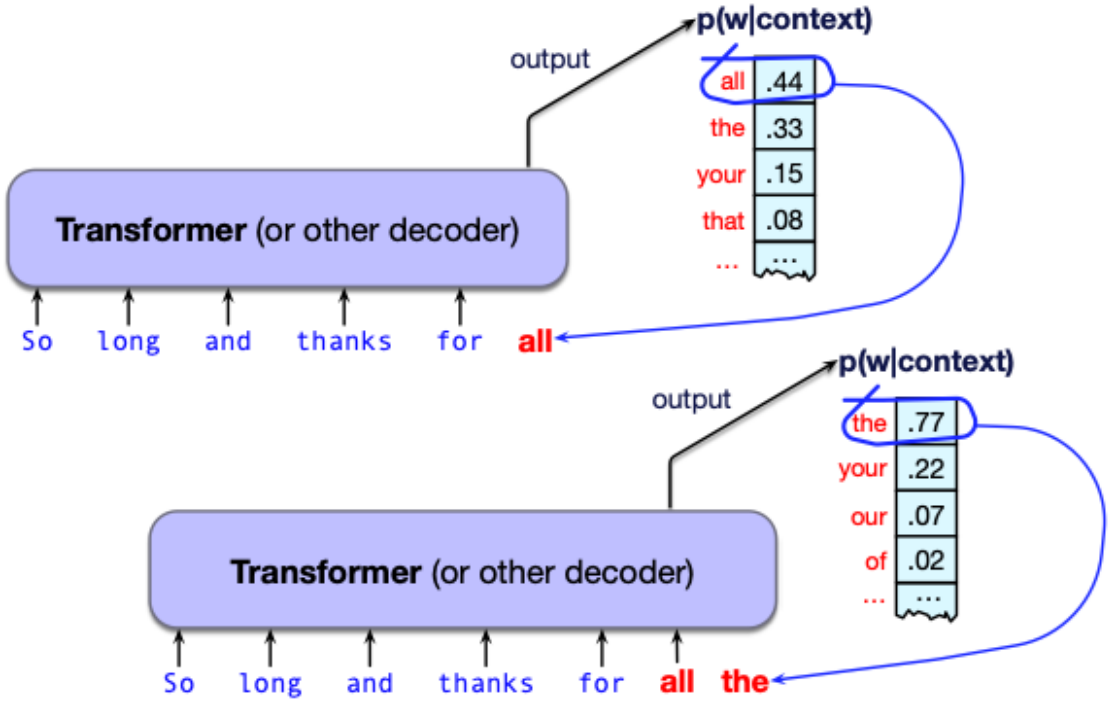
- Attention is learned way of deciding which other words matter

Encoders, Decoders and LLMs

What Is a Large Language Model?

- A neural network with
- **Input:** a context or prefix
- **Output:** a probability distribution over possible next words

- Generate text by sampling from that distribution, one token at a time

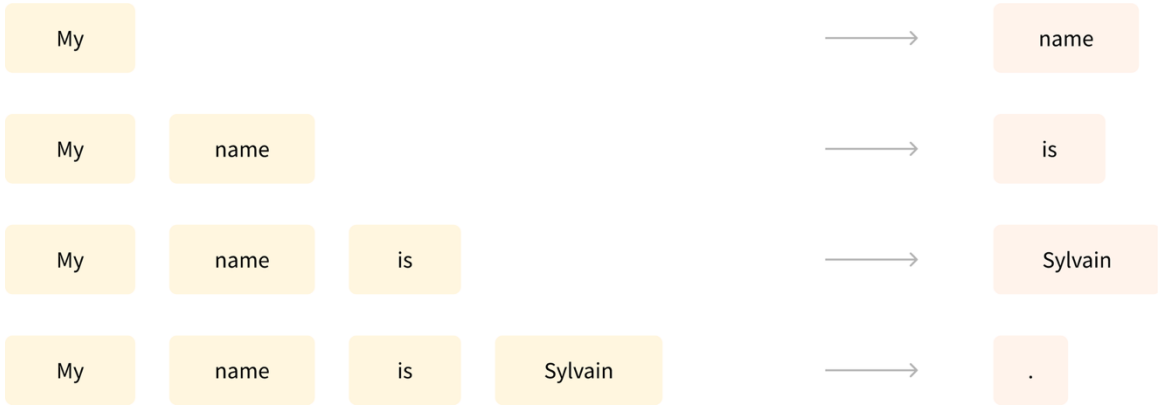


Adapted from Jurafsky, D., & Martin, J. H. (2026). Speech and Language Processing (3rd ed. draft).

Encoders vs. Decoders

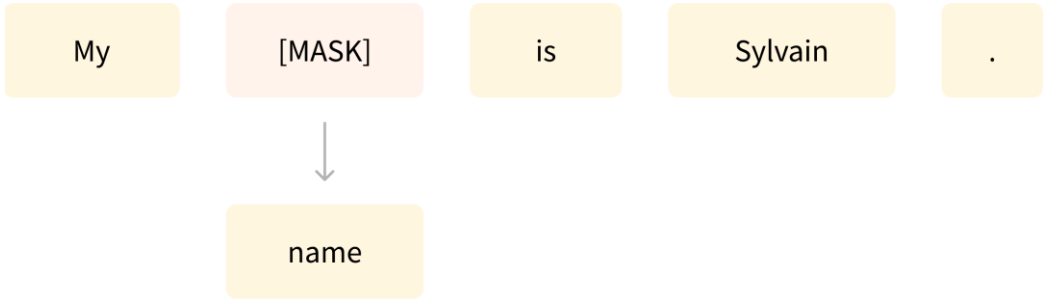
Decoder (GPT, Qwen): predict the next word

only from the words before



Encoder (BERT): predict a masked word

from both sides



Encoders: usually **fine-tuned** with labeled data for classification

Decoders: many tasks **without** extra training, through prompting

Hugging Face LLM Course, ch. 1.4 “How do Transformers work?” (Apache 2.0). huggingface.co/learn/llm-course/chapter1/4

Encoder, Decoder, Encoder-Decoder

	Encoder	Decoder	Encoder-decoder
Reads	the whole text, in both directions	only the words to the left	input with an encoder, writes output with a decoder
Trained to	fill in masked words	predict the next word	turn one text into another
Output	a vector per token / per text	new text, token by token	new text from input text
Typical use	fine-tuning for classification, embeddings	chat, zero-shot prompting, annotation	translation, summarization
Examples	BERT, RoBERTa, sentence-transformers	GPT, Claude, Llama, Qwen	T5, Flan-T5, Whisper
Online harm	fine-tuned hate speech classifiers, BERTopic	LLMs as annotators	translating posts before classifying

Prompting

Zero-Shot Prompting: Classification Without Examples

Prompt

- Classify the text into neutral, negative or positive.
- Text: I think the movie was not good not bad.
- Sentiment:

Model output

- Neutral

- Only an instruction, no labeled examples

Plaza-del-Arco et al. (2023): Zero-Shot Hate Speech Detection

Problem

- Little labeled data, and hate speech varies across contexts and languages

Idea: zero-shot prompting

- Ask a language model whether a text is hateful, without training it on labeled examples
- The answer words (“verbalizers”, e.g. respectful / toxic) are part of the prompt design

Setup and findings

- Several language models and verbalizers, 8 benchmark datasets, 3 languages
- The prompt matters a lot; recent LLMs can match or beat fine-tuned models: promising for under-resourced languages

Plaza-del-Arco, F. M., Nozza, D., & Hovy, D. (2023). Respectful or toxic? Using zero-shot learning with language models to detect hate speech. Proceedings of the 7th Workshop on Online Abuse and Harms (WOAH), 60–68.

Few-Shot Prompting: A Handful of Examples in the Prompt

Task description	Classify the text into one of the following emotions: “anger”, “fear”, “sadness”, “joy”:
Examples	The movie was so scary → fear The children really enjoy the gifts → joy The chef is mad at the customers → anger My mum didn’t let me go to the concert → sadness
Text to classify	Next week I go to a super party! → ?

Which Model for Which Job?

- **Many labeled posts, large data, low cost per post:** fine-tune an encoder (BERT family)
- **Few or no labels, a new category, fast prototyping:** prompt a decoder LLM (zero-shot or few-shot)
- **Finding themes, clustering, similarity:** sentence embeddings from an encoder (e.g., BERTopic)

- Whatever you choose: **validate against human labels** (precision, recall, F1, agreement)

Your Turn: An Open-Source LLM as Annotator

Take-Aways

- **TF-IDF** turns text into numbers; **logistic regression** learns one weight per word from the labels
- **Neural networks** stack many logistic-regression-like units and learn their own features; all are trained with a loss function and gradient descent
- **Embeddings** are dense vectors learned from context: similar meaning → nearby vectors
- Embeddings are also a **measurement tool** for social science (Garg et al. 2018)
- **Attention** builds contextual embeddings , e.g., “it” depend on the sentence
- **Encoders** (BERT) represent and classify, **decoders** (GPT, Qwen) generate; most LLMs are decoders
- Whatever the model: **validate** against human labels

Before Next Week (19 November)

- Colab session (bring a laptop)
- Topics might change
- **Project idea (and group members) due 19 November**
- For inspiration: “Applied political science papers” and the list of data sources on GitHub
- I will add resources and tutorials to the GitHub over the weekend, so check from time to time
- Project presentation is a graded part of the final examination; it is necessary to pass the course!

What would you like to have covered in the practical coding sessions? (Think about whether this is also useful for the rest of the class.)

Do not edit
How to change the design



Audience Q&A

ⓘ The Slido app must be installed on every computer you're presenting from

slido

Universität Konstanz

Questions?
Thank you!

Jun.-Prof. Dr. Miriam Schirmer
Political and Administrative Science

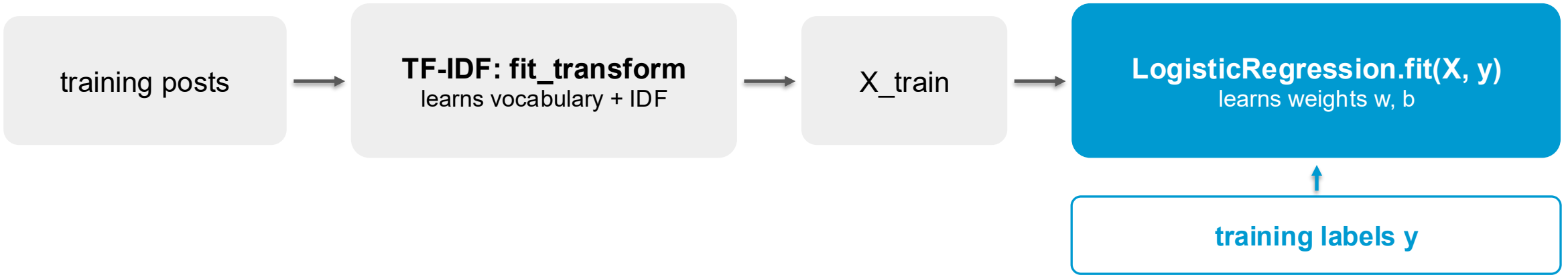
miriam.schirmer@uni-konstanz.de

Credits: Parts of this slide deck have been adapted from slides created by Jurafsky & Martin, Debora Nozza, and Julia Mendelsohn.

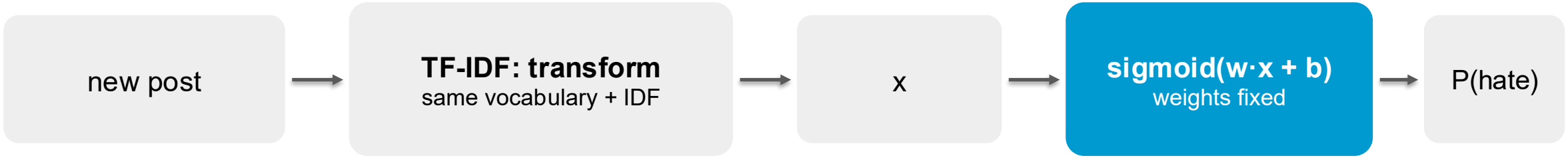
Appendix

What Gets Passed Along?

Training

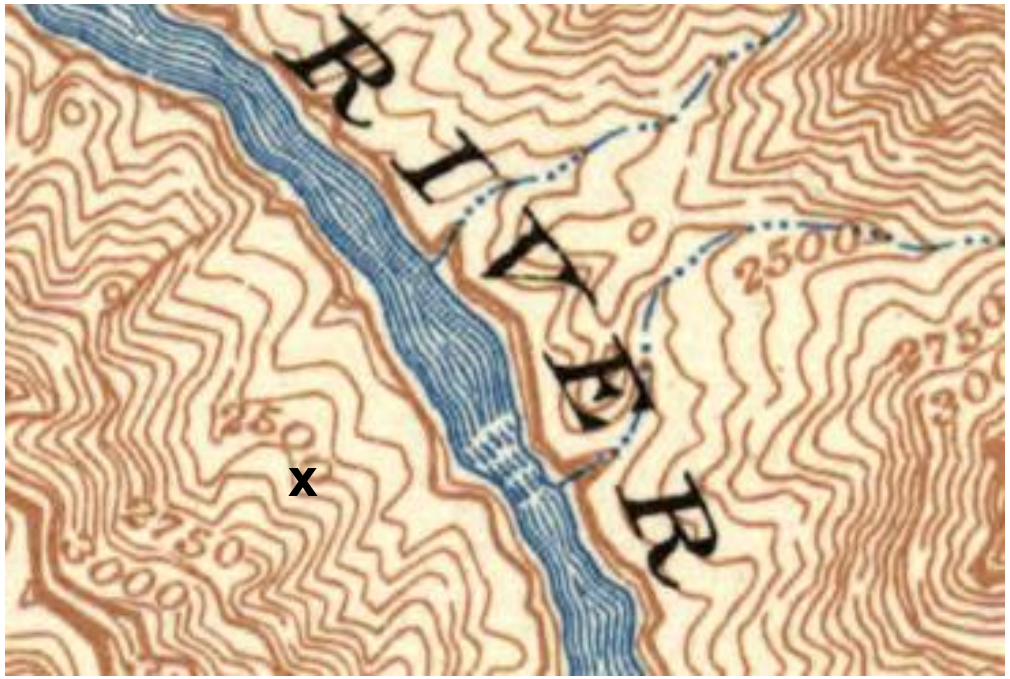


Prediction



Intuition of Gradient Descent

How do I get to the bottom of this river canyon?



- Look around 360°
- Find the direction of steepest slope down
- Go that way

Read more (tutorial text):
https://medium.com/@guptamuskaan438/gradient-descent-explained-visually-from-intuition-to-optimization-f20a40798af4

Adapted from Jurafsky, D., & Martin, J. H. (2026). Speech and Language Processing (3rd ed. draft).

Gradient Descent

- The gradient of a function is a vector pointing in the direction of greatest increase
- **Gradient descent**: Find the gradient of the loss function at the current point and move in the opposite direction
  - Gradient descent calculates the gradient from the entire training dataset → *very slow*
  - **Stochastic gradient descent** is more widely used (update after a small batch of samples)
- The **learning rate** hyperparameter governs how much you move in that direction
  - Too high: you overshoot the minimum
  - Too low: super slow

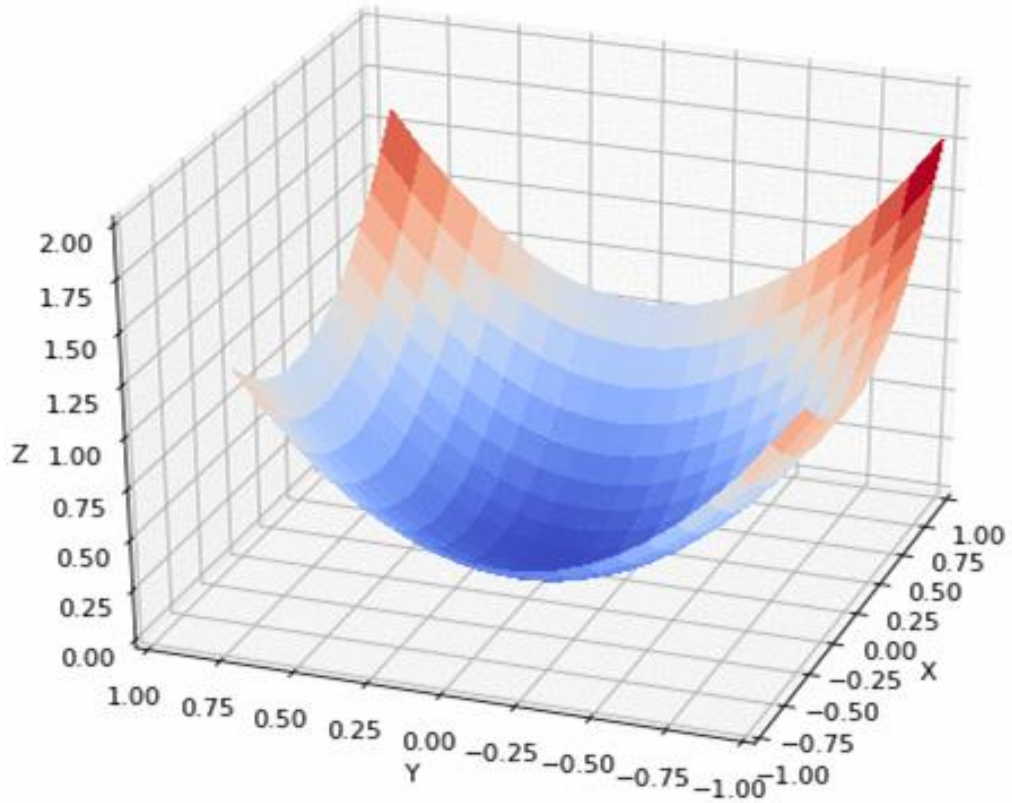
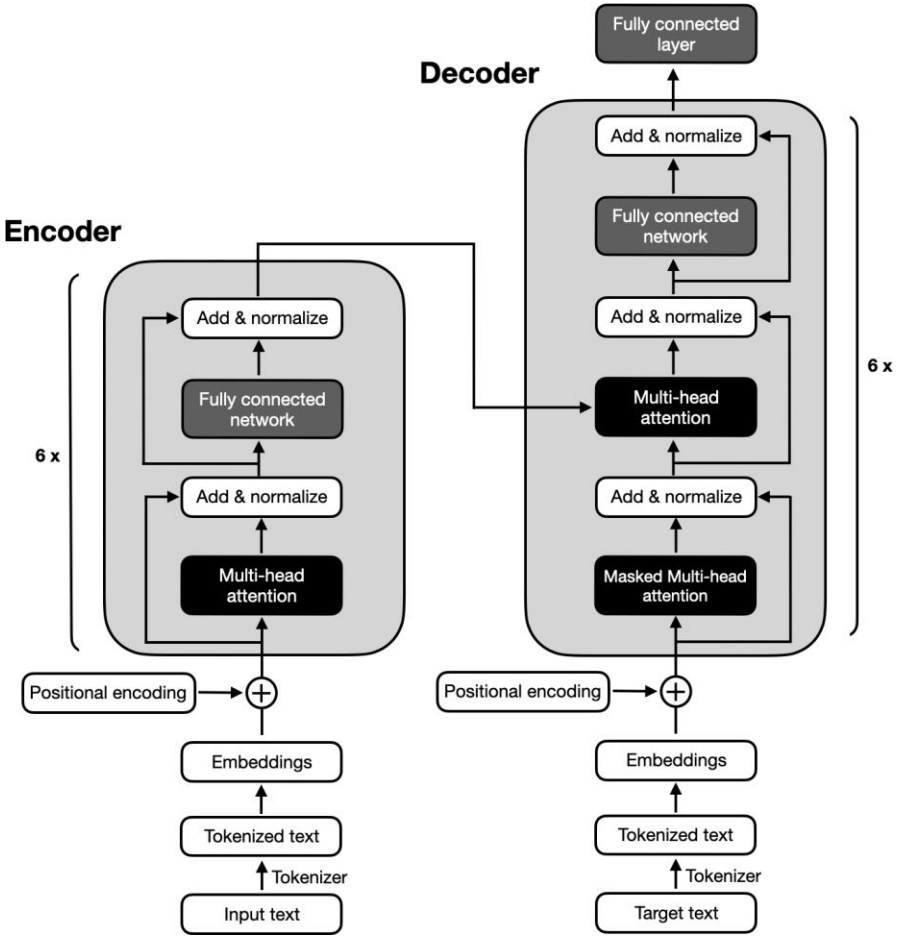


Figure: Mendelsohn, J. (2026). INST664, Week 7: Text Classification. University of Maryland (course slides).

Encoder and Decoder Inside the Transformer



Encoder (left)

reads the whole input; every word can attend to every other word → contextual embeddings

BERT = only the encoder

Decoder (right)

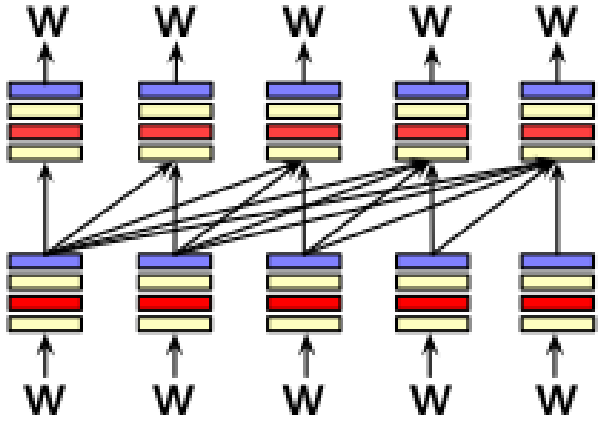
masked attention: each word only sees the words before it → generates text word by word

GPT, Llama, Qwen = only the decoder

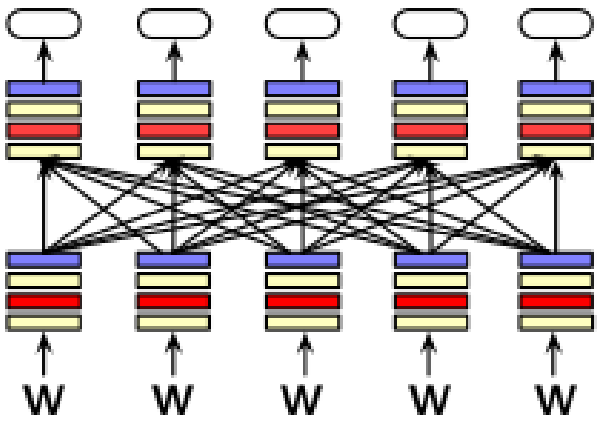
The original transformer (2017) used both, for translation

Figure: Raschka, S. (2023). Understanding encoder and decoder LLMs. Ahead of AI. magazine.sebastianraschka.com/p/understanding-encoder-and-decoder

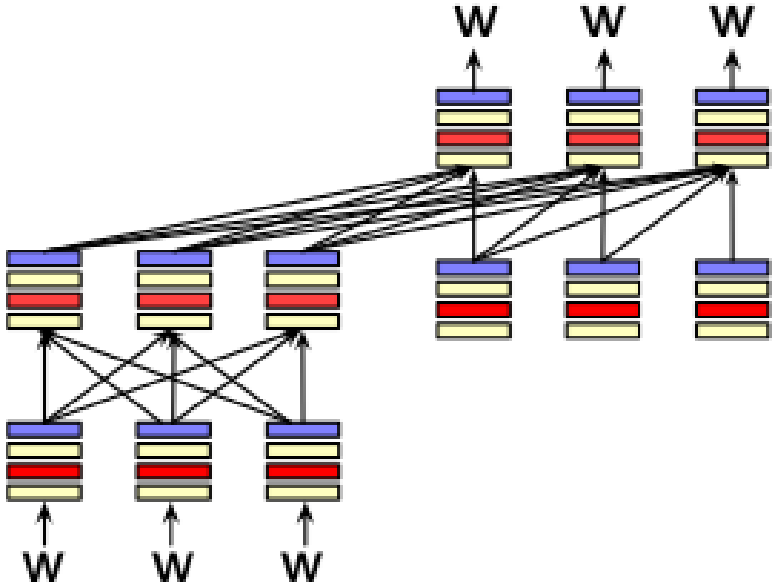
Three Architectures



Decoders
GPT, Claude, Llama, Qwen



Encoders
BERT and family



Encoder-decoders
Flan-T5, Whisper

Adapted from Jurafsky, D., & Martin, J. H. (2026). Speech and Language Processing (3rd ed. draft).

Temperature, Top-k and Top-p Sampling

Temperature

- Reshapes the probability distribution over the next token
- Temperature 1: distribution unchanged; close to 0: the most likely token almost always wins
- Even at 0, results are not always fully deterministic (GPU arithmetic)

Top-k and top-p

- **Top-k:** only the k most likely tokens are considered
- **Top-p (nucleus):** the smallest set of tokens whose probabilities add up to p

- For annotation: no sampling (greedy decoding), as in today's notebook (do_sample=False)